

simple way to know if we are in an entry that is near the antidiagonal: the entries just above the antidiagonal will have indices that sum to $n-2$, and the entries just below the antidiagonal will have indices that sum to n . We'll want to implement Rule 3 in these exceptional cases:

```
if(where_to_go == 'up_left' and (entry_i + entry_j) == (n)):
    new_entry_i = entry_i - 1
    new_entry_j = entry_j - 1
    square[new_entry_i][new_entry_j] = rule3(square[entry_i][entry_j],n,True)

if(where_to_go == 'down_right' and (entry_i + entry_j) == (n-2)):
    new_entry_i = entry_i + 1
    new_entry_j = entry_j + 1
    square[new_entry_i][new_entry_j] = rule3(square[entry_i][entry_j],n,False)
```

Keep in mind that our magic square is finite, so we cannot, for example, travel up/left from the top row or leftmost column. By creating our list of where it's possible to travel based on our current location, we can add some simple logic to ensure that we travel only in allowed directions:

```
where_we_can_go = []

if(entry_i < (n - 1) and entry_j < (n - 1)):
    where_we_can_go.append('down_right')

if(entry_i < (n - 1) and entry_j > 0):
    where_we_can_go.append('down_left')

if(entry_i > 0 and entry_j < (n - 1)):
    where_we_can_go.append('up_right')

if(entry_i > 0 and entry_j > 0):
    where_we_can_go.append('up_left')
```

We have all the elements we need to write Python code that implements Kurushima's algorithm.

Putting It All Together

We can put everything together in a function that takes a starting square with some `nan` entries and travels through it using our three rules to fill them